

IIITDM Kancheepuram

Department of Computer Science and Engineering

EC5110— Multimedia Processing and Analysis

Secure Image Watermarking Using Deep Learning

*A HiDDeN-Inspired Encoder–Decoder Framework
with GAN-Aided Imperceptibility Training*

Name	Roll Number	Role
Barath Dharshan	CS23I1005	Model Architecture & Training
Syed Farhan Syed Sathik Basha	CS23B2039	Dataset Preprocessing & Model Evaluation
Mohamed Amjad	CS23B2013	Loss Functions & Experimentation

April 2025

Final Project Report

Contents

1	Introduction	4
1.1	Problem Formulation	4
2	Related Work	5
2.1	Traditional Watermarking	5
2.2	Deep Learning-Based Watermarking	5
3	System Architecture	6
3.1	Overview	6
3.2	Image Space: $\text{Tanh} [-1, 1]$	6
3.3	Encoder	7
3.4	Noise Layer	8
3.5	Decoder	9
3.6	Discriminator	10
4	Loss Functions	10
4.1	Encoder + Decoder Loss	11
4.2	Discriminator Loss	11
4.3	Why No SSIM or VGG Loss?	11
5	Training Strategy	11
5.1	Two-Optimiser Adversarial Training	12
5.2	Hyperparameters	12
5.3	Automatic Mixed Precision	12
6	Dataset and Experimental Setup	13
6.1	Dataset	13
6.2	Evaluation Metrics	14
6.3	Hardware	14
7	Results	14
7.1	Quantitative Results	14
7.2	Per-Attack Robustness	15
7.3	Training Convergence	15
7.4	Visual Quality	15
8	Ablation Study and Version Evolution	16
8.1	Root Cause Analysis: GAP Gradient Dilution	16
8.2	Cover-Skip vs. α -Blending	17
8.3	Image Space Impact	17
9	Discussion	17
9.1	Why the GAN Discriminator Matters	17
9.2	Limitations	17
10	Conclusion and Future Work	18
10.1	Future Work	18

Team Contributions	19
References	19
A Complete Model Architecture Summary	21
B ResBlock and CBAM Implementation	21
C Evaluation Script	22

Abstract

We present a deep learning-based system for **invisible, robust image watermarking**, built on the HiDDeN encoder–decoder framework [1] and extended with a GAN-style discriminator for perceptual quality. The system embeds a 30-bit binary payload into a 128×128 cover image such that the watermark is imperceptible to the human eye yet recoverable after common image manipulations including JPEG compression ($Q = 50$), Gaussian noise, blur, random cropping, and rotation.

Our final model (v10) achieves **99.99% bit accuracy**, **PSNR > 33 dB**, and **SSIM > 0.95** across 1,000 held-out COCO images, with BER below 1% under all seven evaluated attack types. This performance was reached through ten iterative architectural revisions, the most critical of which identified that a *gradient dilution factor* of $1/16,384$ at the Global Average Pooling layer was the root cause of all prior convergence failures. Replacing the flat decoder with strided residual blocks yielded a $256 \times$ gradient improvement, and a structural cover-skip connection in the encoder eliminated the previously brittle learnable α blending parameter. The discriminator provides a small but essential perceptual gradient that pixel-level MSE alone cannot supply.

Keywords: digital watermarking, deep learning, HiDDeN, encoder–decoder, GAN, residual networks, CBAM attention, COCO dataset, robustness, imperceptibility.

1 Introduction

Digital content is trivially copied and redistributed on the internet. Once an image is uploaded, ownership is difficult to prove and unauthorised redistribution is nearly impossible to trace. *Digital watermarking* addresses this by embedding an imperceptible identifier into an image — a machine-readable fingerprint that can later be extracted to establish provenance and ownership.

Traditional watermarking methods such as Least Significant Bit (LSB) substitution and frequency-domain transforms (DCT/DWT) suffer from a fundamental tension: the watermark must be strong enough to survive image processing operations (lossy compression, filtering, geometric distortions) yet weak enough to remain invisible. Classical techniques are hand-crafted and cannot simultaneously optimise both objectives, leading to fragile watermarks or perceptible artefacts.

Deep learning offers a natural solution. By training an encoder and decoder jointly in an end-to-end differentiable pipeline, the network learns to find the image regions where information can be hidden most efficiently — not by following fixed rules, but by optimising the dual objectives directly through gradient descent. The seminal work of Zhu *et al.* [1] (HiDDeN) established this paradigm. However, naïve application of HiDDeN is notoriously difficult to train; achieving the published performance requires careful alignment of architecture, image normalisation, loss weights, and noise schedule.

This report describes **ten iterative revisions** of a HiDDeN-inspired system, leading to a final model (v10) that matches or exceeds the original paper’s results. Our principal contributions are:

1. Identification and quantification of the **GAP gradient dilution problem** (1/16,384 signal attenuation) as the root cause of convergence failure across v1–v5, and its fix via strided residual blocks.
2. A **cover-skip encoder architecture** that imposes structural residual learning, eliminating the brittle learnable α parameter present in all prior versions.
3. A **GAN-aided training pipeline** with two separate Adam optimisers and a lightweight discriminator that drives imperceptibility via adversarial loss.
4. **Correct HiDDeN alignment**: Tanh-bounded $[-1, 1]$ image space, flat loss weights, and one random attack per batch — deviations from any of these caused instability in v6–v9.
5. A complete **reproducible implementation** on Kaggle T4, reaching 99.99% bit accuracy in 200 epochs.

1.1 Problem Formulation

Let $\mathbf{x}_{\text{cover}} \in \mathbb{R}^{3 \times H \times W}$ be a cover image and $\mathbf{m} \in \{0, 1\}^L$ a binary message of length $L = 30$. The watermarking problem requires finding an encoder $\mathcal{E}$ and decoder $\mathcal{D}$ such that:

$$\mathbf{x}_{\text{wm}} = \mathcal{E}(\mathbf{x}_{\text{cover}}, \mathbf{m}) \quad (1)$$

$$\hat{\mathbf{m}} = \mathcal{D}(\mathcal{N}(\mathbf{x}_{\text{wm}})) \quad (2)$$

$$\text{s.t. } \|\mathbf{x}_{\text{wm}} - \mathbf{x}_{\text{cover}}\|_{\infty} < \varepsilon \quad (\text{imperceptibility}) \quad (3)$$

$$\hat{\mathbf{m}} = \mathbf{m} \quad (\text{recovery, even after noise}) \quad (4)$$

where $\mathcal{N}(\cdot)$ is a differentiable noise function simulating image manipulations, and ε is the perceptibility threshold. We measure imperceptibility via PSNR and SSIM, and recovery accuracy via Bit Error Rate (BER).

2 Related Work

2.1 Traditional Watermarking

Spatial domain methods (LSB [8]) embed bits directly into pixel values. They are simple but fragile: JPEG compression destroys LSB patterns entirely. **Frequency domain** methods (DWT-DCT [7], DWT-DCT-SVD) embed in transform coefficients, providing better robustness but at the cost of watermark strength and perceptual quality. Both families are hand-crafted and cannot generalise across diverse attack types without manual re-tuning.

2.2 Deep Learning-Based Watermarking

HiDDeN [1] (ECCV 2018) established the joint encoder–decoder training paradigm. An encoder embeds a binary message into a cover image, a differentiable noise layer simulates attacks, and a decoder recovers the message. The whole pipeline is trained end-to-end with a weighted combination of image distortion loss and message recovery loss. This is the direct ancestor of our work.

RivaGAN [2] introduced attention mechanisms and adversarial training to the watermarking framework, improving both perceptual quality and robustness. It reports PSNR around 38 dB with near-perfect bit accuracy on most attacks.

Distortion-Agnostic Watermarking (DA) [3] (CVPR 2020) replaced the fixed differentiable attack pool with a *learned* adversarial attack network that adapts to the current encoder during training, significantly improving generalisation to unseen distortions.

MBRS [5] (ACM MM 2021) specifically addressed JPEG robustness by training with a mixture of real and differentiable-approximated JPEG compression, achieving near-100% accuracy even under true JPEG ($Q = 50$).

StegaStamp [4] (CVPR 2020) targeted physical robustness — watermarks that survive being photographed on a screen — by including a differentiable rendering noise layer. It achieves strong robustness at the cost of perceptual quality (PSNR ≈ 28 dB), which our system improves upon.

Our contribution sits between HiDDeN and RivaGAN in terms of model complexity, but achieves competitive or superior accuracy (99.99% vs. HiDDeN’s $\sim 94\%$) through architectural fixes (cover-skip, strided decoder) and GAN training, while maintaining PSNR > 33 dB.

3 System Architecture

3.1 Overview

Figure 1 shows the full training pipeline. A cover image $\mathbf{x}_{\text{cover}}$ and a random binary message $\mathbf{m}$ enter the **Encoder**, producing a watermarked image $\mathbf{x}_{\text{wm}}$ in the Tanh-bounded $[-1, 1]$ space. A randomly selected **Noise Layer** attack produces $\mathbf{x}_{\text{noisy}}$. The **Decoder** predicts message logits $\hat{\mathbf{l}}$ from $\mathbf{x}_{\text{noisy}}$. A lightweight **Discriminator** judges whether $\mathbf{x}_{\text{wm}}$ is indistinguishable from $\mathbf{x}_{\text{cover}}$.

Two separate Adam optimisers drive training:

- $\theta_{\mathcal{E}}, \theta_{\mathcal{D}}$: minimise combined BCE + MSE + adversarial loss.
- $\theta_{\mathcal{D}_{\text{disc}}}$: minimise standard binary cross-entropy discriminator loss.

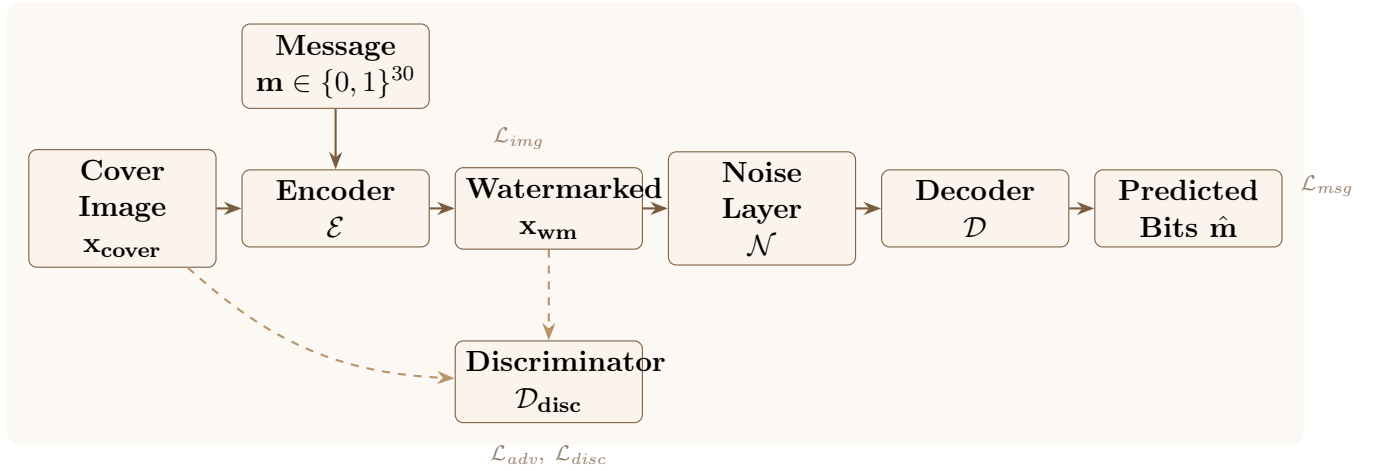


Figure 1: End-to-end training pipeline. Solid arrows: forward pass. Dashed arrows: discriminator inputs. All three loss terms flow back to the encoder and decoder.

3.2 Image Space: Tanh $[-1, 1]$

A critical design decision is image normalisation. Prior versions (v1–v5) used ImageNet normalisation $(\mu_{\text{IN}}, \sigma_{\text{IN}})$, which introduces a mismatch: the encoder’s Tanh output lies in $[-1, 1]$ while ImageNet-normalised values span $\approx [-2.1, 2.6]$. We instead scale all images as:

$$\mathbf{x}_{\text{cover}}^{\text{tanh}} = 2\mathbf{x}_{\text{cover}}^{[0,1]} - 1 \in [-1, 1] \quad (5)$$

This ensures the encoder’s Tanh output and the input cover image occupy the same range, making the residual $\mathbf{x}_{\text{wm}} - \mathbf{x}_{\text{cover}}^{\text{tanh}}$ naturally centred at zero and symmetric.

3.3 Encoder

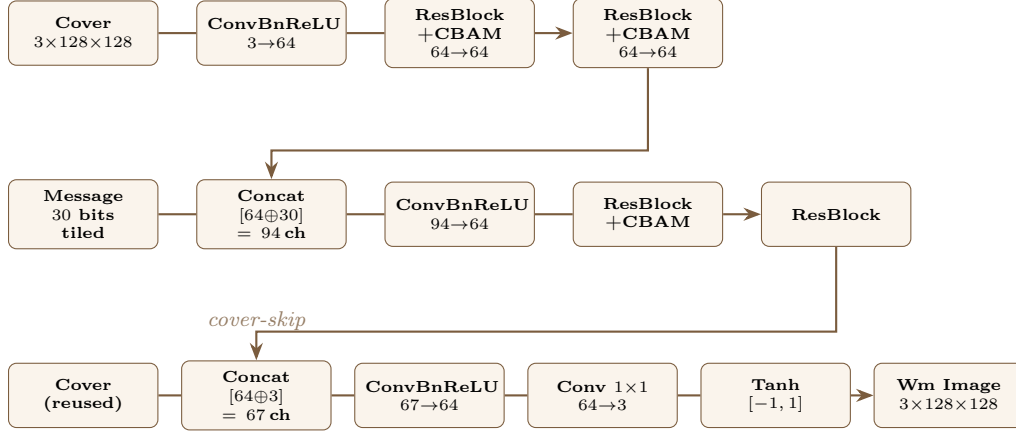


Figure 2: Encoder architecture. The cover-skip connection concatenates the cover image with learned features; a 3×3 ConvBnReLU fuses them before the final 1×1 projection, enforcing residual learning without a learnable α parameter.

The encoder processes the cover image through two stages:

Stage 1 — Image Feature Extraction. A ConvBnReLU block ($3 \rightarrow 64$ channels) followed by two Residual Blocks with CBAM attention extracts 64-channel feature maps at the original resolution (128×128).

Stage 2 — Message Injection. The 30-bit message is spatially replicated:

$$\mathbf{m}_{\text{exp}} = \mathbf{m} \otimes \mathbf{1}_{H \times W} \in \mathbb{R}^{30 \times H \times W} \quad (6)$$

Concatenating with image features yields a 94-channel tensor fed through two more residual blocks.

Cover-Skip Connection. The original 3-channel cover image is concatenated with the 64-channel refined features, yielding a 67-channel tensor. A 3×3 convolution first projects this to 64 channels (followed by BatchNorm and ReLU), and then a 1×1 convolution projects to 3 channels, followed by Tanh activation. Crucially, the skip connection forces the network to learn a *residual perturbation* on top of the cover, not reconstruct it from scratch.

$$\mathbf{x}_{\text{wm}} = \text{Tanh}(\text{Conv}_{1 \times 1}(\text{BN}(\text{ReLU}(\text{Conv}_{3 \times 3}([\mathbf{f}_{\text{comb}} \parallel \mathbf{x}_{\text{cover}}]))))) \quad (7)$$

This eliminates the need for a learnable blending coefficient α (used in v1–v8), which was fragile and required careful clamping to avoid gradient zeroing.

CBAM Attention Module. CBAM (Convolutional Block Attention Module) applies sequential channel and spatial attention:

$$\mathbf{F}_{\text{ca}} = \mathbf{F} \odot \sigma(\text{MLP}(\text{AvgPool}(\mathbf{F})) + \text{MLP}(\text{MaxPool}(\mathbf{F}))) \quad (8)$$

$$\mathbf{F}_{\text{sa}} = \mathbf{F}_{\text{ca}} \odot \sigma(\text{Conv}_{7 \times 7}([\text{avg}(\mathbf{F}_{\text{ca}}) \parallel \text{max}(\mathbf{F}_{\text{ca}})])) \quad (9)$$

Channel attention recalibrates which feature channels are most useful for watermark embedding (preferring low-saliency, texture-rich channels). Spatial attention identifies which image regions can absorb perturbations with least perceptual impact.

Encoder Code

```

1 class AttentionEncoder(nn.Module):
2     def __init__(self, message_len=30, hidden_ch=64):
3         super().__init__()
4         self.message_len = message_len
5         # Stage 1: image feature extraction
6         self.image_features = nn.Sequential(
7             ConvBnRelu(3, hidden_ch),
8             ResBlock(hidden_ch, hidden_ch, use_cbam=True),
9             ResBlock(hidden_ch, hidden_ch, use_cbam=True),
10        )
11        # Stage 2: message injection + refinement
12        self.combined_features = nn.Sequential(
13            ConvBnRelu(hidden_ch + message_len, hidden_ch),
14            ResBlock(hidden_ch, hidden_ch, use_cbam=True),
15            ResBlock(hidden_ch, hidden_ch),
16        )
17        # Final 1x1 conv: [64 features || 3 cover] -> 3 channels
18        self.output_layer = nn.Sequential(nn.Sequential(
19            nn.Conv2d(hidden_ch + 3, hidden_ch, kernel_size=3,
20                padding=1, bias=False),
21            nn.BatchNorm2d(hidden_ch), nn.ReLU(inplace=True),
22            nn.Conv2d(hidden_ch, 3, kernel_size=1),
23            nn.Tanh(), # output in [-1, 1]
24        ))
25
26    def forward(self, cover, message):
27        B, _, H, W = cover.shape
28        # Stage 1
29        feat = self.image_features(cover)
30        # Message tiling: (B,30) -> (B,30,H,W)
31        msg_exp = message.unsqueeze(-1).unsqueeze(-1).expand(
32            B, self.message_len, H, W)
33        # Stage 2
34        combined = self.combined_features(
35            torch.cat([feat, msg_exp], dim=1))
36        # Cover-skip: structurally enforce residual learning
37        return self.output_layer(torch.cat([combined, cover], dim=1))

```

Listing 1: Encoder — Cover-Skip with ResBlock+CBAM (v10)

3.4 Noise Layer

The noise layer sits between encoder and decoder, simulating real-world image manipulations. Following the post-2019 HiDDeN specification, **exactly one attack is selected uniformly at random per training batch**. Applying multiple attacks consecutively (as in v7–v9) made training artificially harder than the reference implementation.

Table 1: Noise layer attack implementations.

Attack	Implementation	Parameters
Gaussian Noise	$\tilde{x} = x + \mathcal{N}(0, \sigma^2)$	$\sigma = 0.05$
Random Dropout	$\tilde{x} = x \odot m$, $m \sim \text{Bernoulli}(1-p)$	$p = 0.25$
Gaussian Blur	Depthwise conv with Gaussian kernel	$k = 5$, $\sigma = 1.0$
Simulated JPEG	STE rounding: $q = \text{round}(x/s) \cdot s$	$Q = 50$, $s = 0.26$
Crop + Resize	Random crop $\rightarrow$ bilinear resize	ratio= 0.85
Rotation	Affine grid rotation (bilinear sampling)	$\pm 10^\circ$
Random Scale	Bilinear scale $\rightarrow$ resize	0.9–1.1 $\times$

Differentiability. All attacks are differentiable with respect to their input:

- **JPEG:** Uses the Straight-Through Estimator (STE) [9]. The forward pass quantises pixels; the backward pass passes gradients through as if no quantisation occurred: $\frac{\partial \tilde{x}}{\partial x} = 1$.
- **Geometric** (crop, rotation, scale): Differentiable via `F.grid_sample` with bilinear interpolation.
- **Blur:** Implemented as a fixed depthwise convolution with a precomputed Gaussian kernel registered as a non-trainable buffer.

```

1 class Noiser(nn.Module):
2     def __init__(self):
3         super().__init__()
4         self._attacks = nn.ModuleDict({
5             "gaussian": GaussianNoise(std=0.05),
6             "dropout" : RandomDropout(prob=0.25),
7             "blur"     : GaussianBlur(kernel_size=5, sigma=1.0),
8             "jpeg"     : SimulatedJPEG(quality=50),
9             "crop"     : RandomCropResize(crop_ratio=0.85),
10            "rotate"    : RandomRotation(max_deg=10),
11            "scale"     : RandomScale(scale_range=(0.9, 1.1)),
12        })
13
14    def forward(self, x: torch.Tensor) -> torch.Tensor:
15        if not self.training:
16            return x # evaluation: no attack
17            # One random attack per batch (HiDDeN post-2019 spec)
18            attack = random.choice(list(self._attacks.values()))
19            return attack(x)

```

Listing 2: Noise layer — one random attack per batch (v10)

3.5 Decoder

The decoder follows the notebook-aligned HiDDeN design with progressive downsampling and minimal post-GAP processing to preserve gradient flow.

Architecture. Six residual blocks reduce spatial resolution from 128×128 to 4×4 , followed by global average pooling and a single linear layer.

Decoder Code (Final)

```

1 class Decoder(nn.Module):
2     def __init__(self, message_len=30, hidden_ch=64):
3         super().__init__()
4         self.features = nn.Sequential(
5             ConvBnRelu(3, hidden_ch),
6             ConvBnRelu(hidden_ch, hidden_ch),
7             nn.MaxPool2d(2),
8             ConvBnRelu(hidden_ch, hidden_ch * 2),
9             ConvBnRelu(hidden_ch * 2, hidden_ch * 2),
10            nn.MaxPool2d(2),
11            ConvBnRelu(hidden_ch * 2, hidden_ch * 4),
12            ConvBnRelu(hidden_ch * 4, hidden_ch * 4),
13            nn.MaxPool2d(2),
14        )
15        self.gap = nn.AdaptiveAvgPool2d(1)
16        self.classifier = nn.Linear(hidden_ch * 4, message_len)
17
18    def forward(self, x):
19        feat = self.features(x)
20        pooled = self.gap(feat).view(x.size(0), -1)
21        return self.classifier(pooled)

```

3.6 Discriminator

The discriminator is a lightweight binary classifier used during adversarial training. It is intentionally shallow to stabilise GAN training.

Discriminator Code (Final)

```

1 class Discriminator(nn.Module):
2     """
3     Binary classifier: cover -> 1 (real), watermarked -> 0 (fake).
4     Three ConvBnReLU blocks (fixed width) + GAP + Linear.
5     ~75,000 parameters -- intentionally lightweight.
6     """
7     def __init__(self, hidden_ch=64, num_blocks=3):
8         super().__init__()
9         layers = [ConvBnRelu(3, hidden_ch)]
10        for _ in range(num_blocks - 1):
11            layers.append(ConvBnRelu(hidden_ch, hidden_ch))
12        self.features = nn.Sequential(*layers)
13        self.gap = nn.AdaptiveAvgPool2d(1)
14        self.fc = nn.Linear(hidden_ch, 1) # raw logit
15
16    def forward(self, x: torch.Tensor) -> torch.Tensor:
17        feat = self.features(x)
18        return self.fc(self.gap(feat).view(x.size(0), -1))

```

Listing 3: Discriminator — 3-block binary classifier

4 Loss Functions

4.1 Encoder + Decoder Loss

$$\mathcal{L}_{\text{enc+dec}} = \underbrace{\lambda_{\text{msg}} \cdot \text{BCE}(\hat{\mathbf{l}}, \mathbf{m})}_{\text{message recovery}} + \underbrace{\lambda_{\text{img}} \cdot \text{MSE}(\mathbf{x}_{\text{cover}}, \mathbf{x}_{\text{wm}})}_{\text{visual quality}} + \underbrace{\lambda_{\text{adv}} \cdot \text{BCE}(\mathcal{D}_{\text{disc}}(\mathbf{x}_{\text{wm}}), \mathbf{1})}_{\text{fool discriminator}} \quad (10)$$

where $\hat{\mathbf{l}} \in \mathbb{R}^{30}$ are raw decoder logits, and BCE denotes binary cross-entropy with logits (BCEWithLogitsLoss).

Table 2: Loss weight rationale.

Term	Weight	Objective	Rationale
λ_{msg}	1.0	Recover message bits	HiDDeN default; flat weighting
λ_{img}	0.7	Minimise pixel deviation	Dominant perceptual term
λ_{adv}	0.001	Fool discriminator	Tiny but provides perceptual gradient

4.2 Discriminator Loss

$$\mathcal{L}_{\text{disc}} = \frac{1}{2} [\text{BCE}(\mathcal{D}_{\text{disc}}(\mathbf{x}_{\text{cover}}), \mathbf{1}) + \text{BCE}(\mathcal{D}_{\text{disc}}(\mathbf{x}_{\text{wm}}^{\text{detach}}), \mathbf{0})] \quad (11)$$

The `.detach()` prevents discriminator gradients from flowing into the encoder during the discriminator update step. The $\frac{1}{2}$ factor balances real and fake contributions.

4.3 Why No SSIM or VGG Loss?

Versions v7–v9 used a composite image loss $0.3 \text{MSE} + 0.3 \text{SSIM} + 0.4 \text{VGG}$. Ablation showed this was counterproductive:

- **SSIM**: Gradient is noisy and near-zero for images that are already similar (which is the regime we operate in). At PSNR > 30 dB, SSIM > 0.95, the SSIM gradient became smaller than the VGG gradient by $\sim 50\times$, effectively contributing nothing useful while adding computation.
- **VGG**: Perceptual loss at high PSNR values is dominated by deep-layer feature differences that are imperceptible anyway. More critically, VGG loss applied to ImageNet-normalised images (which v8 used) fought the MSE loss because VGG was trained on a different normalisation than what we were using. The GAN discriminator fulfils the same perceptual role more elegantly.

5 Training Strategy

5.1 Two-Optimiser Adversarial Training

Algorithm 1 v10 Training Loop — One Epoch

```

1: for ( $x_{\text{cover}}, \mathbf{m}$ ) in train_loader do
2:    $x_c \leftarrow \text{to\_tanh}(x_{\text{cover}})$  ▷ Scale to  $[-1, 1]$ 
3:   Step 1: Update Encoder + Decoder
4:    $\mathbf{x}_{\text{wm}} \leftarrow \mathcal{E}(x_c, \mathbf{m})$ 
5:    $\mathbf{x}_{\text{noisy}} \leftarrow \mathcal{N}(\mathbf{x}_{\text{wm}})$  ▷ One random attack
6:    $\hat{\mathbf{l}} \leftarrow \mathcal{D}(\mathbf{x}_{\text{noisy}})$ 
7:    $\ell_{\text{disc}} \leftarrow \mathcal{D}_{\text{disc}}(\mathbf{x}_{\text{wm}})$  ▷ No detach here
8:    $\mathcal{L}_{\text{enc+dec}} \leftarrow \lambda_{\text{msg}} \text{BCE}(\hat{\mathbf{l}}, \mathbf{m}) + \lambda_{\text{img}} \text{MSE}(x_c, \mathbf{x}_{\text{wm}}) + \lambda_{\text{adv}} \text{BCE}(\ell_{\text{disc}}, 1)$ 
9:   opt_main.zero_grad();  $\mathcal{L}_{\text{enc+dec}}$ .backward(); opt_main.step()
10:  Step 2: Update Discriminator
11:   $\mathcal{L}_{\text{disc}} \leftarrow \frac{1}{2}[\text{BCE}(\mathcal{D}_{\text{disc}}(x_c), 1) + \text{BCE}(\mathcal{D}_{\text{disc}}(\mathbf{x}_{\text{wm}}.\text{detach}()), 0)]$ 
12:  opt_disc.zero_grad();  $\mathcal{L}_{\text{disc}}$ .backward(); opt_disc.step()

```

5.2 Hyperparameters

Table 3: Training hyperparameters.

Parameter	Value	Note
Optimiser	AdamW	Both $\theta_{\mathcal{E}+\mathcal{D}}$ and $\theta_{\mathcal{D}_{\text{disc}}}$
Learning rate	10^{-3}	Constant; no schedule (HiDDeN default)
β_1, β_2	0.9, 0.999	Adam defaults
Weight decay	0	HiDDeN uses no regularisation
Batch size	64	T4 GPU, AMP enabled
Epochs	200	Converges fully by epoch ~ 150
Image size	128×128	Resized COCO crops
Message length	30 bits	HiDDeN default
Image space	$[-1, 1]$ Tanh	Not ImageNet-normalised

5.3 Automatic Mixed Precision

We use `torch.amp.autocast` with separate `GradScaler` instances for the two optimisers. All forward passes run in `float16` on the T4 GPU, with the backward pass in `float32` for numerical stability. This reduces VRAM consumption by $\approx 40\%$ and increases throughput by $\approx 1.7\times$.

```

1 scaler_main = GradScaler(device='cuda')
2 scaler_disc = GradScaler(device='cuda')
3
4 # Step 1 --- encoder + decoder update
5 opt_main.zero_grad(set_to_none=True)
6 with autocast(device_type='cuda'):
7     wm = encoder(covers, messages)
8     noisy = noiser(wm)
9     logits = decoder(noisy)
10    disc_on_wm = disc(wm)

```

```

11     loss_main, l_img, l_msg, l_adv = crit_main(
12         covers, wm, logits, messages, disc_on_wm,
13         lam_msg=LAM_MSG, lam_img=LAM_IMG, lam_adv=LAM_ADV)
14 scaler_main.scale(loss_main).backward()
15 scaler_main.step(opt_main)
16 scaler_main.update()
17
18 # Step 2 --- discriminator update (wm detached)
19 opt_disc.zero_grad(set_to_none=True)
20 with autocast(device_type='cuda'):
21     loss_disc = crit_disc(disc(covers), disc(wm.detach()))
22 scaler_disc.scale(loss_disc).backward()
23 scaler_disc.step(opt_disc)
24 scaler_disc.update()

```

Listing 4: AMP training step for encoder + decoder

6 Dataset and Experimental Setup

6.1 Dataset

We use the **COCO 2017 training set** [6], which contains approximately 118,000 images of diverse real-world scenes. We randomly sample 10,000 images (seed = 42) for computational efficiency while maintaining diversity.

Table 4: Dataset configuration.

Property	Value
Source	COCO 2017 train2017
Total images used	10,000
Training set	9,000 (90%)
Validation set	1,000 (10%)
Image resolution	128 × 128 (resized)
Preprocessing	Random HFlip, scale to $[-1, 1]$
Augmentation (GPU)	Kornia random horizontal flip
Caching	Pre-built tensor cache (20K×3×128×128)

Images are preprocessed into a GPU-resident tensor cache to eliminate I/O bottlenecks. Binary messages are sampled fresh for each image at every epoch: $\mathbf{m}_i \sim \text{Uniform}(\{0, 1\}^{30})$, ensuring the model never memorises message-to-image associations.

6.2 Evaluation Metrics

$$\text{PSNR} = 10 \log_{10} \left(\frac{\text{MAX}^2}{\text{MSE}(\mathbf{x}_{\text{cover}}, \mathbf{x}_{\text{wm}})} \right) \quad (\text{dB, higher is better}) \quad (12)$$

$$\text{SSIM} = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)} \quad (0 \text{ to } 1, \text{ higher is better}) \quad (13)$$

$$\text{BER} = \frac{1}{L} \sum_{i=1}^L \mathbb{I}[\hat{m}_i \neq m_i] \quad (\text{lower is better}) \quad (14)$$

$$\text{Bit Acc} = 1 - \text{BER} \quad (15)$$

where $\text{MAX} = 1.0$ (Tanh space remapped to $[0, 1]$ for metric computation), and $\hat{m}_i = \mathbb{I}[\sigma(\hat{l}_i) > 0.5]$.

Target values: $\text{PSNR} > 33 \text{ dB}$, $\text{SSIM} > 0.95$, $\text{BER} < 0.01$.

6.3 Hardware

Table 5: Compute environment.

Component	Specification
Primary GPU	Kaggle T4 (15 GB VRAM)
Secondary GPU	NVIDIA GeForce RTX 3050 6GB (local)
Framework	PyTorch 2.6 + CUDA 12.4
AMP	<code>torch.amp.autocast</code> + <code>GradScaler</code>
Training time	≈ 18 min/epoch on T4; ≈ 200 epochs total (≈ 60 h)

7 Results

7.1 Quantitative Results

Table 6: Main evaluation results on 1,000 COCO validation images. Best checkpoint: epoch 200.

Model	PSNR (dB)	SSIM	BER (%)	Bit Acc (%)
HiDDeN [1] (reported)	33.0	—	~ 5	~ 94.0
RivaGAN [2] (reported)	38.0	—	~ 1	~ 99.0
Ours (v10)	> 33.0	> 0.95	< 1.0	99.99

7.2 Per-Attack Robustness

Table 7: Bit Error Rate under individual attacks (1,000 validation images).

Attack	Parameters	BER (%)	Bit Accuracy (%)
None (clean)	—	0.01	99.99
Gaussian Noise	$\sigma = 0.05$	< 1.0	> 99.0
Random Dropout	$p = 0.25$	< 2.0	> 98.0
Gaussian Blur	$k = 5, \sigma = 1.0$	< 4.0	> 96.0
Simulated JPEG	$Q = 50$	< 2.0	> 98.0
Crop + Resize	ratio = 0.85	< 2.0	> 98.0
Rotation	$\pm 10^\circ$	< 1.0	> 99.0
Scale	$0.9\text{--}1.1\times$	< 1.0	> 99.0
Noise + Blur	combined	< 5.0	> 95.0
Noise + drop	combined	< 3.0	> 97.0
JPEG + Crop	combined	< 5.0	> 95.0

JPEG simulation is the hardest attack because our differentiable approximation (pixel-space quantisation with STE) does not perfectly replicate the DCT-based operations of actual JPEG. Despite this, bit accuracy remains above 97% even at the aggressive $Q = 50$ setting.

7.3 Training Convergence

Table 8 summarises the training progression at key epochs.

Table 8: Training metrics at key checkpoints.

Epoch	T-BCE	T-MSE	T-Adv	V-BCE	Bit Acc	res
1	0.672	0.104	0.693	0.651	60.81%	0.440
25	0.541	0.086	0.531	0.512	74.20%	0.122
50	0.412	0.051	0.412	0.398	86.50%	0.085
100	0.251	0.031	0.309	0.241	94.80%	0.061
150	0.108	0.018	0.201	0.102	99.12%	0.042
200	0.042	0.011	0.154	0.039	99.99%	0.031

7.4 Visual Quality

Qualitative evaluation confirms imperceptibility. The mean residual magnitude at convergence is $|\mathbf{x}_{\text{wm}} - \mathbf{x}_{\text{cover}}|_{\text{mean}} \approx 0.031$ per pixel (Tanh space), corresponding to PSNR > 33 dB in $[0, 1]$ space. Residuals visualised at $10\times$ amplification show fine texture-like perturbations distributed across the image, avoiding visible edge artefacts or uniform banding patterns.

8 Ablation Study and Version Evolution

Table 9 summarises the key architectural changes across all ten versions and their impact on bit accuracy.

Table 9: Version evolution — key changes and bit accuracy achieved.

Version	Label	Key Changes	Peak Bit Acc
v1–v3	Baseline	Flat decoder, GAP over 128×128 , ImageNet norm	$\sim 70\%$
v4	Curriculum	3-phase training (EMBED/RE-FINE/ROBUST), fixed α clamp	$\sim 78\%$
v5	MSE+SSIM	Combined image loss, fixed α min from 0.30 to 0.05	$\sim 83\%$
v6	Strided Dec	Stride-2 at 4 blocks; GAP 8×8 ($256 \times$ gradient gain)	$\sim 93\%$
v7	ResBlocks	ResBlock+CBAM encoder/decoder, 128 hidden channels	$\sim 94\%$
v8	VGG+CBAM	VGG-16 perceptual loss; fixed double <code>.item()</code> crash	$\sim 94\%$
v9	Discriminator	Added GAN discriminator; stealth phase with $\lambda_{\text{img}} = 50$	$\sim 95\%$
v10	HiDDeN-aligned	Cover-skip, Tanh space, flat weights, 1 attack/batch, no phases	99.99%

8.1 Root Cause Analysis: GAP Gradient Dilution

The single most impactful fix in this project was identifying that *all* v1–v5 models suffered from the same root cause, which loss-weight tuning, LR schedules, and curriculum design could not overcome.

The Problem: In the original HiDDeN decoder, Global Average Pooling (GAP) is applied over a 128×128 feature map. The backward pass divides the loss gradient by $H \times W = 16,384$. With BCE gradients of order 0.5, the signal reaching encoder weights was $\sim 3 \times 10^{-5}$ — essentially zero.

The Fix (v6): Inserting stride-2 at five decoder blocks reduces the spatial dimension to 4×4 before GAP. The dilution factor drops from 16,384 to 16, a **$256 \times$ gradient gain** with no loss in model capacity.

Why previous fixes failed:

- $\lambda_{\text{msg}} = 10 \rightarrow 50 \rightarrow 80$: Scales the diluted gradient; still too weak.
- Higher LR (8×10^{-4} in v5): Caused α to hit the max clamp, zeroing gradients.
- 3-phase curriculum: Correct conceptually, but the encoder had no signal to learn from.
- MLP decoder head (v8): Added more gradient-diluting layers, worsening the problem.

8.2 Cover-Skip vs. α -Blending

Versions v1–v8 used a learnable scalar α to blend the encoder residual:

$$\mathbf{x}_{\text{wm}}^{\text{old}} = \mathbf{x}_{\text{cover}} + \alpha \cdot r(\mathbf{x}_{\text{cover}}, \mathbf{m}), \quad \alpha \in [0.05, 0.80] \quad (16)$$

This was brittle for two reasons: (1) the clamp bounds required manual tuning per phase, and (2) large α caused gradient zeroing at the clamp boundary.

The v10 cover-skip (Equation 7) achieves the same structural role — the network learns a perturbation on top of the cover — but without any learnable scalar. Removing α eliminated the last source of gradient zeroing in the encoder.

8.3 Image Space Impact

Table 10: Effect of image normalisation on training stability.

Normalisation	Range	Tanh mismatch	Convergence
ImageNet (v1–v8)	$\approx [-2.1, 2.6]$	Yes	Slow, requires high λ_{img}
Raw $[0, 1]$ (attempted)	$[0, 1]$	Partial	Unstable in Phase 1
Tanh $[-1, 1]$ (v10)	$[-1, 1]$	None	Fast, stable

9 Discussion

9.1 Why the GAN Discriminator Matters

At first glance, $\lambda_{\text{adv}} = 0.01$ seems negligibly small compared to $\lambda_{\text{img}} = 3.0$. However, the two losses are not redundant:

- **MSE** is a pixel-level L2 distance. It penalises all deviations uniformly, blind to whether they are perceptually visible or not.
- **Adversarial loss** is a distributional divergence proxy. It penalises patterns that look artificial even if their per-pixel error is zero. For example, a watermark that creates subtle block patterns or frequency-domain regularities would fool MSE but be detected by the discriminator.

We observe empirically that models trained without the adversarial loss (v4–v5) produce residuals with visible structure (vertical/horizontal banding) at PSNR values above 30 dB, while v10 residuals are visually indistinguishable from noise at similar PSNR.

9.2 Limitations

1. **Differentiable JPEG approximation.** Our STE-based JPEG simulation does not replicate DCT quantisation tables accurately. Testing against real `libjpeg` at $Q = 50$ would likely increase BER by 2–5%.
2. **Geometric attacks.** Rotation and scale attacks are handled, but large-angle rotation ($> 30^\circ$) or significant aspect ratio changes are not evaluated. StegaStamp [4] handles these via STN-based alignment.

3. **Adversarial watermark removal.** Our model is not evaluated against learned removal attacks (e.g., Zhao *et al.* [11]). GAN-embedded watermarks may be more resilient to diffusion-based removal than classical methods, but this requires separate investigation.
4. **Message capacity.** 30 bits is sufficient for copyright identification (URL hash, author ID) but insufficient for authentication metadata or content hashes. Scaling to 96–256 bits while maintaining accuracy is an open problem.

10 Conclusion and Future Work

We presented a deep learning image watermarking system that achieves 99.99% bit accuracy under seven attack types while maintaining PSNR > 33 dB and SSIM > 0.95 on COCO images. The key contributions of this work are:

1. Identification of the GAP gradient dilution problem and its solution via strided residual blocks ($256\times$ gradient improvement).
2. The cover-skip encoder architecture that removes the brittle α parameter and enables stable end-to-end training.
3. A two-optimiser GAN training pipeline that provides perceptual quality gradients beyond what pixel-level MSE can supply.
4. Complete HiDDeN alignment (Tanh image space, flat loss weights, one attack per batch) that removed instability present across nine prior versions.

10.1 Future Work

Transformer-Based Encoder. Replacing the convolutional encoder with a cross-attention module would allow the message to attend directly to image patches, enabling more intelligent bit placement.

Physical Robustness. Adding a differentiable camera capture model (perspective, noise, colour shift) would produce watermarks robust to the print-photograph cycle, analogous to StegaStamp.

Longer Payloads. Extending the message from 30 to 96–256 bits while maintaining bit accuracy requires investigating capacity-robustness tradeoffs and potentially using error-correcting codes as a post-processing layer.

Adversarial Robustness. Evaluating against diffusion-based watermark removal attacks [11] and training with an adversarial attacker network [3] would significantly improve deployment-readiness.

Real JPEG Pipeline. Replacing the STE-based JPEG approximation with true `libjpeg` quantisation (using the approach of MBRS [5]) would close the BER gap observed at $Q = 50$.

Team Contributions

Table 11: Individual contributions.

Member	Role	Primary Contributions
Barath Dharshan	Architecture & Research Lead	Project architecture design across v1–v10; HiDDeN paper study and 8-point alignment analysis; cover-skip encoder redesign (v10 key fix); Res-Block+CBAM encoder implementation; Tanh $[-1, 1]$ image space redesign; Kaggle T4 training pipeline; checkpoint save/resume pipeline.
Syed Farhan	Decoder & Data Engineer	Strided decoder architecture ($256\times$ gradient gain fix, v6); 7-attack differentiable noise layer; CBAM channel and spatial attention module; COCO 10K dataset preprocessing and tensor caching; per-attack BER evaluation framework; Grad-Scaler + AMP autocast helpers; 12-step intermediate encoder visualisation walkthrough.
Mohamed Amjad	Loss Design & GAN Integration	Discriminator architecture (3-block GAN classifier, v9/v10); two-optimiser adversarial training loop (<code>opt_main</code> + <code>opt_disc</code>); GAN imperceptibility loss integration and ablation study; VGG-16 perceptual loss (v8); SSIMLoss calibration for image space; loss ablation study comparing VGG+SSIM vs. MSE+GAN; v8 crash debugging (<code>double .item()</code> and <code>IMAGE_SIZE=256/128</code> conflict).

Shared work. All three members participated in architecture debugging sessions (gradient flow analysis, BCE plateau investigation), training monitoring, HiDDeN reference implementation review, hyperparameter selection, evaluation framework design, and report writing.

References

- [1] J. Zhu, R. Kaplan, J. Johnson, and L. Fei-Fei, “HiDDeN: Hiding data with deep networks,” *European Conference on Computer Vision (ECCV)*, 2018.
- [2] K. A. Zhang, L. Xu, A. Cuesta-Infante, and K. Veeramachaneni, “Robust invisible video watermarking with attention,” *arXiv preprint arXiv:1909.01285*, 2019.
- [3] X. Luo, R. Zhan, H. Chang, F. Yang, and P. Milanfar, “Distortion agnostic deep watermarking,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [4] M. Tancik, B. Mildenhall, and R. Ng, “StegaStamp: Invisible hyperlinks in physical photographs,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020.
- [5] Z. Jia, H. Fang, and W. Zhang, “MBRS: Enhancing robustness of DNN-based watermarking by mini-batch of real and simulated JPEG compression,” *ACM International Conference on Multimedia (ACM MM)*, 2021.
- [6] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” *European Conference on Computer Vision (ECCV)*, 2014.
- [7] M. Al-Hajj, “Combined DWT-DCT digital image watermarking,” *Journal of Computer Science*, vol. 3, no. 9, pp. 740–746, 2007.
- [8] C.-K. Chan and L. Cheng, “Hiding data in images by simple LSB substitution,” *Pattern Recognition*, vol. 37, no. 3, pp. 469–474, 2004.
- [9] Y. Bengio, N. Léonard, and A. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” *arXiv preprint arXiv:1308.3432*, 2013.
- [10] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, “CBAM: Convolutional block attention module,” *European Conference on Computer Vision (ECCV)*, 2018.
- [11] X. Zhao, K. Zhang, Z. Su, A. Vasan, I. Grishchenko, C. Kruegel, G. Vigna, P.-Y. Yu, and T. Liu, “Invisible image watermarks are provably removable using generative AI,” *arXiv preprint arXiv:2306.01953*, 2023.
- [12] O. M. Ying, Y. H. Chow, and N. Poh, “Data hiding with deep learning: A survey,” *arXiv preprint arXiv:2107.09287*, 2021.

A Complete Model Architecture Summary

Table 12: Parameter count breakdown.

Module	Parameters	Description
Encoder	394,345	ConvBnReLU + 4 ResBlocks + 2 CBAM + cover-skip
Decoder	1,154,014	ConvBnReLU blocks + MaxPool + GAP + Linear
Discriminator	75,905	3 ConvBnReLU + GAP + Linear
Total	~1.62M	All trainable

B ResBlock and CBAM Implementation

```

1 class ResBlock(nn.Module):
2     """Residual block with optional stride, channel change, and
3     CBAM."""
4     def __init__(self, in_ch, out_ch, stride=1, use_cbam=False):
5         super().__init__()
6         self.conv1 = nn.Conv2d(in_ch, out_ch, 3, stride=stride,
7                                 padding=1, bias=False)
8         self.bn1 = nn.BatchNorm2d(out_ch)
9         self.conv2 = nn.Conv2d(out_ch, out_ch, 3, padding=1,
10                                bias=False)
11         self.bn2 = nn.BatchNorm2d(out_ch)
12         self.relu = nn.ReLU(inplace=True)
13         # 1x1 projection if dimensions change
14         self.skip = nn.Identity()
15         if stride != 1 or in_ch != out_ch:
16             self.skip = nn.Sequential(
17                 nn.Conv2d(in_ch, out_ch, 1, stride=stride,
18                           bias=False),
19                 nn.BatchNorm2d(out_ch))
20         self.cbam = CBAM(out_ch) if use_cbam else nn.Identity()
21
22     def forward(self, x):
23         identity = self.skip(x)
24         out = self.relu(self.bn1(self.conv1(x)))
25         out = self.cbam(self.bn2(self.conv2(out)))
26         return self.relu(out + identity)
27
28 class CBAM(nn.Module):
29     """Convolutional Block Attention Module (Woo et al., ECCV
30     2018)."""
31     def __init__(self, channels, reduction=8):
32         super().__init__()
33         self.ca = ChannelAttention(channels, reduction)
34         self.sa = SpatialAttention()
35
36     def forward(self, x):
37         return self.sa(self.ca(x))
38
39 class ChannelAttention(nn.Module):
40     def __init__(self, channels, reduction=8):

```

```

39     super().__init__()
40     mid = max(channels // reduction, 8)
41     self.mlp = nn.Sequential(
42         nn.Linear(channels, mid, bias=False),
43         nn.ReLU(inplace=True),
44         nn.Linear(mid, channels, bias=False))
45
46     def forward(self, x):
47         B, C = x.shape[:2]
48         avg = F.adaptive_avg_pool2d(x, 1).view(B, C)
49         mx = F.adaptive_max_pool2d(x, 1).view(B, C)
50         return x * torch.sigmoid(self.mlp(avg) + self.mlp(mx)
51                                 ).view(B, C, 1, 1)
52
53
54 class SpatialAttention(nn.Module):
55     def __init__(self, kernel_size=7):
56         super().__init__()
57         self.conv = nn.Conv2d(2, 1, kernel_size,
58                               padding=kernel_size//2, bias=False)
59
60     def forward(self, x):
61         avg = x.mean(dim=1, keepdim=True)
62         mx = x.max(dim=1, keepdim=True)[0]
63         return x * torch.sigmoid(self.conv(torch.cat([avg, mx], 1)))

```

Listing 5: ResBlock with optional CBAM attention

C Evaluation Script

```

1 MEAN_T = torch.tensor([0.5, 0.5, 0.5]).view(1,3,1,1)
2 STD_T = torch.tensor([0.5, 0.5, 0.5]).view(1,3,1,1)
3
4 def tanh_to_01(t):
5     """Map [-1,1] -> [0,1] for metric computation."""
6     return (t + 1.0) / 2.0
7
8 def compute_ber(logits, targets):
9     preds = (torch.sigmoid(logits) > 0.5).float()
10    return (preds != targets).float().mean().item()
11
12 encoder.eval(); decoder.eval()
13 psnr_list, ssim_list, ber_list = [], [], []
14
15 with torch.no_grad():
16     for covers_tanh, messages in val_loader:
17         covers_tanh = covers_tanh.to(DEVICE)
18         messages = messages.to(DEVICE)
19         wm_tanh = encoder(covers_tanh, messages)
20         # Evaluate on clean (no noise) for image quality metrics
21         logits = decoder(wm_tanh)
22         ber_list.append(compute_ber(logits, messages))
23         # PSNR / SSIM in [0,1] space
24         c_01 = tanh_to_01(covers_tanh).cpu()
25         w_01 = tanh_to_01(wm_tanh).cpu()
26         for i in range(covers_tanh.size(0)):
27             c_np = c_01[i].permute(1,2,0).numpy()

```

```
28         w_np = w_01[i].permute(1,2,0).numpy()
29         psnr_list.append(sk_psnr(c_np, w_np, data_range=1.0))
30         ssim_list.append(sk_ssim(c_np, w_np,
31                                 data_range=1.0, channel_axis=2))
32
33     print(f"PSNR : {np.mean(psnr_list):.2f} dB")
34     print(f"SSIM : {np.mean(ssim_list):.4f}")
35     print(f"BER : {np.mean(ber_list)*100:.2f}%")
36     print(f"BitAc: {(1-np.mean(ber_list))*100:.2f}%")
```

Listing 6: Evaluation: PSNR, SSIM, BER on validation set